

Projekt implementacji algorytmu CHAMELEON

Metody Eksploracji Danych — Etap 1

Oliwier Szypczyński

2026-04-26

Spis treści

1	Dokumentacja projektu implementacji — CHAMELEON	2
1.1	Status dokumentu	3
1.2	Spis treści	3
1.3	Kompilacja do PDF	3
1.4	Repozytorium kodu	3
2	1. Wprowadzenie	4
2.1	1.1. Cel projektu	4
2.2	1.2. Problem klasteryzacji hierarchicznej	4
2.3	1.3. Miejsce CHAMELEON-a w taksonomii algorytmów klasteryzacji	4
2.4	1.4. Motywacja: “dynamic modeling”	5
2.5	1.5. Kontekst projektu i deliverables	6
2.6	1.6. Struktura niniejszego dokumentu	6
3	2. Algorytm CHAMELEON	6
3.1	2.1. Idea ogólna	6
3.2	2.2. Schemat dwufazowy	6
3.3	2.3. Faza 0: budowa grafu k-najbliższych sąsiadów (§4.2)	7
3.4	2.4. Faza I: rekurencyjna bisekcja (§4.4 Phase I)	7
3.5	2.5. Faza II: aglomeracyjne łączenie (§4.4 Phase II)	8
3.6	2.6. Miary podobieństwa międzyklastrowego	8
3.7	2.7. Parametry algorytmu — podsumowanie	9
3.8	2.8. Charakterystyka algorytmu	9
4	3. Struktura implementacji	10
4.1	3.1. Architektura ogólna	10
4.2	3.2. Moduł graph.py — §4.2 budowa grafu k-NN	10
4.3	3.3. Moduł partition.py — §4.4 Phase I rekurencyjna bisekcja	13
4.4	3.4. Moduł metrics.py — §4.3 miary podobieństwa	14
4.5	3.5. Moduł merger.py — §4.4 Phase II aglomeracja	15
4.6	3.6. Moduł chameleon.py — klasa estymatora	16
4.7	3.7. Moduł pomocniczy _types.py	17
4.8	3.8. Punkt wejścia pakietu — __init__.py	17

4.9	3.9. Decyzje projektowe	18
5	4. Struktury danych	18
5.1	4.1. Dane źródłowe (wejście)	18
5.2	4.2. Struktury wewnętrzne algorytmu	20
5.3	4.3. Decyzje projektowe — uzasadnienie wyborów	22
5.4	4.4. Podsumowanie — tabela wszystkich aliasów typów	23
6	5. Ocena efektywności proponowanego rozwiązania	23
6.1	5.1. Złożoność teoretyczna	23
6.2	5.2. Benchmarki referencyjne (Moonpuck)	25
6.3	5.3. Predykcja przyspieszeń vs Moonpuck	26
6.4	5.4. Ograniczenia proponowanego rozwiązania	26
6.5	5.5. Podsumowanie	27
7	6. Plan testów	27
7.1	6.1. Narzędzia	27
7.2	6.2. Struktura katalogu tests/	28
7.3	6.3. Testy jednostkowe	28
7.4	6.4. Testy integracyjne (end-to-end)	29
7.5	6.5. Test zgodności sklearn API	30
7.6	6.6. Testy porównawcze z implementacją referencyjną	30
7.7	6.7. Testy parametryczne (sweep)	31
7.8	6.8. Testy skalowalności	32
7.9	6.9. Pokrycie kodu (coverage)	32
7.10	6.10. CI (Continuous Integration)	32
7.11	6.11. Podsumowanie planu testów	33
8	7. Zbiory danych	33
8.1	7.1. Zbiory referencyjne (vendored w tests/data/)	33
8.2	7.2. Zbiory syntetyczne (fixtures)	35
8.3	7.3. Podsumowanie	35
9	8. Bibliografia	36
9.1	Artykuły naukowe	36
9.2	Wymagania projektowe	36
9.3	Implementacje referencyjne	36
9.4	Biblioteki i narzędzia	36

1 Dokumentacja projektu implementacji — CHAMELEON

Autor: Oliwier Szypczyński **Przedmiot:** Metody Eksploracji Danych (MED), semestr letni 2025/2026
Prowadzący: dr inż. Robert Bembenik **Wydział:** Politechnika Warszawska, Wydział Elektroniki i Technik Informacyjnych **Temat:** CHAMELEON: hierarchical clustering algorithm using dynamic modeling (Karypis G., Han E., Kumar V., IEEE Computer 32(8), 1999, str. 68–75)

1.1 Status dokumentu

Niniejszy dokument stanowi **projekt implementacji** algorytmu CHAMELEON — wymagany deliverable Etapu 1 projektu MED z deadline’em 08.05.2026. Zgodnie z wymaganiami prowadzącego (Projekt_MED_2026l_rb.pdf, str. 1–2), dokument zawiera:

- strukturę implementacji z odniesieniem do konkretnych kroków algorytmu (rozdz. 3),
- struktury danych dla danych źródłowych i algorytmu (rozdz. 4),
- ocenę efektywności proponowanego rozwiązania (rozdz. 5).

“Projekt implementacji nie jest implementacją” — *Projekt_MED_2026l_rb.pdf*

Pełna implementacja jest deliverablem Etapu 2 (deadline 02.06.2026).

1.2 Spis treści

Nr	Plik	Treść
1	01_wprowadzenie.md	Cel projektu, problem klasteryzacji hierarchicznej, miejsce CHAMELEON-a w taksonomii
2	02_algorytm.md	Charakterystyka CHAMELEON: 2 fazy, k-NN graph, miary RI/RC/ α
3	03_implementacja.md	Struktura implementacji — funkcje, klasy, mapping na kroki algorytmu
4	04_struktury_danych.md	Struktury danych źródłowe i wewnętrzne algorytmu
5	05_efektywnosc.md	Ocena efektywności — złożoność teoretyczna i benchmarki referencyjne
6	06_plan_testow.md	Plan testów: jednostkowe, integracyjne, porównawcze, parametryczne, skalowalności
7	07_zbiory_danych.md	Charakterystyka zbiorów testowych (Aggregation, smileface, t4_8k, syntetyczne)
8	08_bibliografia.md	Bibliografia

1.3 Kompilacja do PDF

```
bash scripts/build_docs.sh
# tworzy: docs/documentation/dokumentacja.pdf
```

1.4 Repozytorium kodu

Kod źródłowy projektu znajduje się w `src/pychameleon/`. Implementacja będzie publikowana docelowo jako pakiet `pychameleon` na PyPI (lato 2026).

- Repozytorium lokalne: /Users/oszypczy/VSCoDe/studia/MED/
- Stan na dzień oddania Etapu 1: skeleton 5 modułów ze ścisłą zgodnością z sekcjami paperu Karypis 1999 (§4.2, §4.3, §4.4), 12 testów jednostkowych przechodzi, konfiguracja ruff i mypy --strict bez błędów.

2 1. Wprowadzenie

2.1 1.1. Cel projektu

Celem projektu jest opracowanie projektu implementacji oraz docelowo pełnej implementacji hierarchicznego algorytmu klasteryzacji **CHAMELEON** według artykułu źródłowego:

Karypis G., Han E., Kumar V., *CHAMELEON: A hierarchical clustering algorithm using dynamic modeling*, IEEE Computer: Special Issue on Data Analysis and Mining, vol. 32, no. 8, 1999, str. 68–75.

W ramach przedmiotu *Metody Eksploracji Danych* (MED) projekt podzielony jest na dwa etapy:

Etap	Deadline	Punkty	Deliverable
1	08.05.2026	15	Projekt implementacji (niniejszy dokument) + prezentacja założeń
2	02.06.2026	30	Pełna implementacja, eksperymenty, raport końcowy, prezentacja wyników

Niniejszy dokument odpowiada Etapowi 1.

2.2 1.2. Problem klasteryzacji hierarchicznej

Klasteryzacja jest podstawowym zadaniem analizy skupień w eksploracji danych — nieinformacją uczeniem (unsupervised learning), w którym celem jest podział zbioru obiektów $X = \{x_1, \dots, x_n\}$ na rozłączne grupy (klastry) tak, by obiekty w obrębie jednej grupy były do siebie podobne, a obiekty z różnych grup — różne.

Algorytmy klasteryzacji **hierarchicznej** budują dendrogram zagnieżdżonych klastrow, zamiast jednego płaskiego podziału. Wyróżnia się dwa wariant:

- **aglomeracyjny (bottom-up)** — start z n jednoelementowych klastrow; w kolejnych krokach łączy się dwa najbardziej podobne klastry w jeden,
- **podziałowy (top-down)** — start z jednego dużego klastra; w kolejnych krokach dzieli się go na mniejsze.

CHAMELEON należy do rodziny algorytmów aglomeracyjnych, ale różni się od klasycznych podejść (single/complete/average linkage) sposobem **modelowania podobieństwa międzyklastrowego**.

2.3 1.3. Miejsce CHAMELEON-a w taksonomii algorytmów klasteryzacji

Algorytm	Typ	Reprezentacja klastra	Główna miara podobieństwa	Ograniczenia
k-means (1967)	partycjonujący	centroid	odległość euklidesowa	tylko klastry kuliste, ustalone k
DBSCAN (1996)	gęstościowy	w regiony gęstości	ϵ -sąsiedztwo + MinPts	jednolity próg gęstości
CURE (1998)	hierarchiczny	zbiór reprezentantów	minimalna odległość między reprezent.	statyczny model klastra
ROCK (1999)	hierarchiczny	graf "linków"	współdzielone sąsiedztwo	tylko dane kategoryjne; statyczny model
CHAMELEON (1999)	hierarchiczny	sub-klastry z grafu k-NN	$RI \times RC^\alpha$ (dynamiczny model)	wymaga ustawienia k , m , α
OPTICS (1999)	gęstościowy	wykres osiągalności	reachability distance	wymaga eksploracji wykresu
HDBSCAN (2013)	gęstościowy	drzewo gęstości	mutual reachability	brak hierarchii w sensie linkage

CHAMELEON pojawił się jako odpowiedź na ograniczenia algorytmów wcześniejszych, w szczególności CURE i ROCK, które używają **statycznego modelu** klastra (np. zbiór reprezentantów lub macierz podobieństwa kategoryjnego). Statyczny model nie odzwierciedla wewnętrznej struktury klastra ani charakterystyki połączenia między klastrami — co prowadzi do błędnych decyzji o łączeniu w przypadku klastrów o różnych kształtach, gęstościach i rozmiarach.

2.4 1.4. Motywacja: “dynamic modeling”

CHAMELEON wprowadza pojęcie **dynamicznego modelowania klastrów**: kryterium łączenia dwóch klastrów C_i , C_j zależy nie tylko od ich wzajemnego podobieństwa (jak w linkage), ale również od **wewnętrznej charakterystyki** każdego z nich.

Konkretnie, CHAMELEON łączy C_i i C_j wtedy, gdy:

1. ich wzajemne **interconnectivity** (sumaryczna waga krawędzi krzyżujących granicę między klastrami) jest porównywalne z wewnętrzną interconnectivity każdego z nich (tzw. *relative interconnectivity*, RI),
2. **closeness** (średnia waga krawędzi granicznych) jest porównywalna z wewnętrzną closeness każdego z nich (*relative closeness*, RC).

Dzięki temu CHAMELEON poprawnie obsługuje sytuacje, w których:

- klastry mają **różne gęstości** (statyczne miary łączyłyby gęste klastry nawet gdy są wyraźnie odseparowane od rzadkich, co tu się nie dzieje, bo RC normalizuje przez wewnętrzną gęstość),
- klastry mają **różne rozmiary** (RI normalizuje przez wewnętrzną interconnectivity, która rośnie z rozmiarem klastra),
- klastry mają **niewypukłe kształty** (single/average linkage często fragmentuje takie klastry; CHAMELEON utrzymuje je w całości dzięki k-NN grafowi).

Paper Karypis 1999, sekcja 5 (“Experimental Results”), demonstruje tę przewagę na zbiorach syntetycznych (DS1–DS5, gdzie m.in. zbiór t4_8k używany w niniejszym projekcie), na których

k-means, CURE i DBSCAN dają wyraźnie gorsze wyniki.

2.5 1.5. Kontekst projektu i deliverables

Projekt jest realizowany jako pakiet **pychameleon** w nowoczesnej konwencji Python (src-layout, type hints, sklearn API). Długoterminowy cel — niezależny od oceny MED — to publikacja jako biblioteka dostępna przez `pip install pychameleon` (PyPI, lato 2026) oraz potencjalny pull request do `scikit-learn-contrib`.

Wybór nowoczesnych konwencji (sklearn-compatible API od pierwszego commita, src-layout, ruff/mypy w trybie strict, pytest z coverage) jest świadomą decyzją projektową, szczegółowo omówioną w rozdziale 3.

2.6 1.6. Struktura niniejszego dokumentu

- **Rozdział 2** opisuje algorytm CHAMELEON: 2 fazy, definicje miar podobieństwa, pseudokod kluczowych operacji.
- **Rozdział 3** prezentuje **strukturę implementacji** — podział na 5 modułów z jednoznacznym mapowaniem na sekcje paperu Karypis 1999. Jest to najważniejszy rozdział z punktu widzenia wymagań prowadzącego.
- **Rozdział 4** opisuje **struktury danych** — zarówno format danych źródłowych (numpy.ndarray), jak i wewnętrzne struktury algorytmu (CSR adjacency, priority queue).
- **Rozdział 5** zawiera **ocenę efektywności**: złożoność teoretyczną z paragrafu §4.5 paperu oraz empiryczne benchmarki implementacji referencyjnej Moonpuck (3 datasety, 788–8000 punktów).
- **Rozdziały 6–7** dokumentują **plan testów** i **charakterystykę zbiorów testowych**.
- **Rozdział 8** zawiera bibliografię.

3 2. Algorytm CHAMELEON

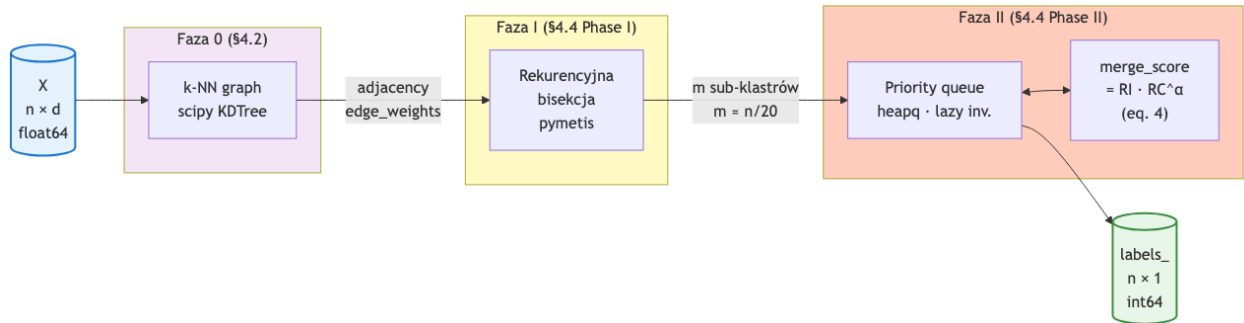
3.1 2.1. Idea ogólna

CHAMELEON jest dwuetapowym algorytmem klasteryzacji hierarchicznej. Zamiast operować bezpośrednio na surowych odległościach między punktami (jak k-means czy linkage), CHAMELEON buduje **graf k-najbliższych sąsiadów** i prowadzi cały proces klasteryzacji jako operacje na tym grafie. Dzięki temu uzyskuje:

- **odporność na różne gęstości**: krawędzie w grafie k-NN łączą każdy punkt z jego najbliższymi sąsiadami niezależnie od gęstości lokalnej,
- **odporność na niewypukłe kształty**: graf k-NN naturalnie podąża za strukturą zbioru,
- **modelowanie wewnętrznej struktury klastra**: po wstępnej fragmentacji grafu na małe podgrafy, każdy z nich ma już zdefiniowaną własną gęstość i interconnectivity.

3.2 2.2. Schemat dwufazowy

Dane wejściowe X (macierz $n \times d$) przechodzą przez trzy etapy: budowę grafu k-najbliższych sąsiadów, rekurencyjną bisekcję (Faza I) oraz aglomeracyjne łączenie sub-klastrów (Faza II). Wynikiem jest wektor `labels_` o długości n przechowujący identyfikator klastra dla każdego punktu.



Rysunek 1: Schemat przepływu danych w algorytmie CHAMELEON

3.3 2.3. Faza 0: budowa grafu k-najbliższych sąsiadów (§4.2)

Cel: zbudować rzadki graf $G = (V, E, w)$, gdzie:

- $V = \{1, \dots, n\}$ — wierzchołki odpowiadają punktom,
- $(i, j) \in E \Leftrightarrow j$ jest jednym z k najbliższych sąsiadów i (lub odwrotnie),
- $w(i, j) = 1 / d(x_i, x_j)$ — waga krawędzi jest **odwrotnością odległości** (większa waga = większe podobieństwo).

Parametr: k_nn (typowo 5–20). Większe k_nn → gęstszy graf → lepsze “przeskakiwanie” przerw między klastrami, ale większy koszt obliczeń.

Implementacja: `scipy.spatial.KDTree.query(X, k=k_nn+1)` w czasie $O(n \log n)$ (pierwszy “sąsiad” to sam punkt, więc bierzemy $k+1$ i odrzucamy).

Reprezentacja w pamięci: lista list (CSR-like adjacency) — szczegóły w rozdz. 4.

3.4 2.4. Faza I: rekurencyjna bisekcja (§4.4 Phase I)

Cel: podzielić graf k-NN na m zwartych sub-klastrów, na których będzie operować faza aglomeracyjna. Sub-klastry są **wstępną granulacją** danych.

Algorytm (pseudokod z paperu Karypis 1999):

```

function PHASE_I(graph G, min_cluster_size MS):
    sub_clusters ← {V(G)}           # początkowo jeden klaster ze wszystkimi wierzchołkami
    while ∃ C ∈ sub_clusters: |C| > MS:
        C ← largest cluster in sub_clusters
        (A, B) ← min_cut_bisect(G[C]) # pymetis: 2-way partition minimizing edge cut
        sub_clusters ← (sub_clusters \ {C}) ∪ {A, B}
    return sub_clusters

```

Implementacja: wywołanie `pymetis.part_graph(2, ...)` w trybie `objtype='cut'`, `ufactor=250` (paper rekomenduje `ufactor=250`, czyli $\pm 25\%$ rozmiarów partycji).

Parametr: `min_cluster_size` (oznaczony w paperu jako `MinSize` lub `MS`). Typowo 2.5% rozmiaru zbioru (`min_cluster_size = 0.025 · n`). Mniejszy `MinSize` → więcej sub-klastrów → drobniejsza granulacja, więcej kandydatów do łączenia w fazie II.

Złożoność: $O(m \cdot \text{pymetis_bisect})$, gdzie $\text{pymetis} \sim O(|E| + |V| \log |V|)$ multilevel METIS (Karypis & Kumar 1998).

3.5 2.5. Faza II: aglomeracyjne łączenie (§4.4 Phase II)

Cel: iteracyjnie łączyć pary sub-klastrów o najwyższym `merge_score` aż do osiągnięcia żądanej liczby klastrów `n_clusters` (lub wyczerpania pozytywnych kandydatów do łączenia).

Algorytm (pseudokod):

```
function PHASE_II(graph G, sub_clusters, target_k k, alpha α):
    queue ← priority_queue()
    for each pair (Ci, Cj) of sub-clusters connected in G:
        score ← RI(Ci, Cj) · RC(Ci, Cj)α
        if score > 0:
            queue.push((-score, Ci, Cj)) # heapq: minus dla max-heap

    while |sub_clusters| > k and queue.not_empty():
        (-score, Ci, Cj) ← queue.pop()
        if Ci or Cj already merged: continue # lazy invalidation
        C□ ← merge(Ci, Cj)
        update queue with neighbors of C□

    return sub_clusters
```

Parametr: α (α). EkspONENT w `merge_score` = $RI \cdot RC^\alpha$ (eq. 4 w paperu):

- $\alpha > 1$ — preferowane łączenia o wysokim **RC** (zachowanie spójności gęstości),
- $\alpha < 1$ — preferowane łączenia o wysokim **RI** (zachowanie sieci połączeń),
- $\alpha = 2.0$ — wartość domyślna z paperu, dobrze sprawdza się na większości zbiorów.

3.6 2.6. Miary podobieństwa międzyklastrowego

Niech $EC_{\{C_i, C_j\}}$ oznacza zbiór krawędzi grafu k -NN łączących wierzchołki klastra C_i z wierzchołkami klastra C_j . Niech EC_{C_i} oznacza krawędzie **bisektora** klastra C_i — czyli zbiór krawędzi, które trzeba usunąć aby podzielić C_i na dwie zbliżone rozmiarem części (uzyskany przez `pymetis.part_graph(2, ...)` na podgrafie indukowanym przez C_i).

3.6.1 2.6.1. Relative Interconnectivity — RI (eq. 1)

$$RI(C_i, C_j) = \frac{|EC_{\{C_i, C_j\}}|}{\frac{1}{2}(|EC_{C_i}| + |EC_{C_j}|)}$$

Interpretacja: stosunek **sumarycznej wagi krawędzi krzyżujących granicę** C_i – C_j do **średniej wewnętrznej interconnectivity** obu klastrów. Wartość $RI \approx 1$ oznacza, że łączenie nie osłabi struktury żadnego klastra.

3.6.2 2.6.2. Relative Closeness – RC (eq. 2)

$$RC(C_i, C_j) = \frac{\bar{S}_{EC_{\{C_i, C_j\}}}}{\frac{|C_i|}{|C_i|+|C_j|} \cdot \bar{S}_{EC_{C_i}} + \frac{|C_j|}{|C_i|+|C_j|} \cdot \bar{S}_{EC_{C_j}}}$$

gdzie $\bar{S}_E$ oznacza **średnią wagę** krawędzi w zbiorze E .

Interpretacja: stosunek **średniej gęstości połączeń granicznych** do **ważonej średniej wewnętrznej gęstości** obu klastrów. Waga jest proporcjonalna do rozmiaru klastra — większy klastier bardziej wpływa na “globalną” gęstość.

3.6.3 2.6.3. Merge score (eq. 4)

$$\text{score}(C_i, C_j) = RI(C_i, C_j) \cdot RC(C_i, C_j)^\alpha$$

To główne kryterium decyzyjne fazy II. Para (C_i, C_j) o najwyższym score zostaje połączona jako pierwsza.

3.6.4 2.6.4. Wyznaczenie EC_{C_i}

EC_{C_i} (interconnectivity wewnętrzna klastra) wymaga **bisekcji** podgrafu klastra C_i . Implementacja używa tej samej funkcji `pymetis.part_graph(2, ...)` co Faza I. Wartość $|EC_{C_i}|$ jest **cache’owana** po pierwszym wyznaczeniu — nie zmienia się dopóki klastier nie zostanie scalony z innym.

3.7 2.7. Parametry algorytmu — podsumowanie

Parametr	Symbol w paperu	Domyślnie	Wpływ
<code>n_clusters</code>	k	8	docelowa liczba klastrów
<code>k_nn</code>	knn	10	liczba sąsiadów w grafie k-NN
<code>min_cluster_size</code>	$MS / MinSize$	$0.025 \cdot n$	minimalny rozmiar sub-klastra w fazie I
<code>alpha</code>	α	2.0	eksponent RC w <code>merge_score</code>

3.8 2.8. Charakterystyka algorytmu

Zalety:

- jakość — wykrywanie klastrów o **różnych kształtach, rozmiarach i gęstościach** (sekcja 5 paperu Karypis 1999 demonstruje to na DS1–DS5, gdzie k-means/CURE/DBSCAN zawodzi),
- elastyczność — graf k-NN można zbudować dla danych o dowolnej metryce odległości (nie tylko euklidesowej),
- generuje pełną hierarchię — można zatrzymać proces na dowolnej liczbie klastrów.

Wady i ograniczenia:

- wymaga ustawienia trzech parametrów (`k_nn`, `min_cluster_size`, `alpha`); wybór jest czuły zwłaszcza dla `k_nn`,
- `pymetis` nie jest deterministyczny — różne uruchomienia mogą dać minimalnie różne bisekcje (omawiane w rozdz. 5),
- naiwna budowa grafu `k-NN` to $O(n^2)$ — w naszej implementacji rozwiązane przez `KDTree` ($O(n \log n)$), patrz rozdz. 5,
- `KDTree` zaprojektowany pod **niskie wymiary** ($\leq 10D$); dla danych wysokowymiarowych zalecamy zastąpienie przez `Annoy` lub `HNSW` w przyszłej wersji.

4 3. Struktura implementacji

Wymaganie #1 z PDF prowadzącego (str. 1): *“strukturze implementacji (funkcje, klasy) z odniesieniem do konkretnych kroków implementowanego algorytmu”*

Niniejszy rozdział mapuje implementację jeden-do-jednego na sekcje paperu Karypis 1999. Każda funkcja i klasa jest opisana z wyraźnym odniesieniem do kroku algorytmu, który realizuje, oraz numeru sekcji w paperu źródłowym.

4.1 3.1. Architektura ogólna

Implementacja jest podzielona na **5 modułów** w katalogu `src/pychameleon/`:

Moduł	Sekcja paperu	Faza algorytmu	Główne funkcje / klasy
<code>graph.py</code>	§4.2	Faza 0	<code>knn_graph(X, k)</code>
<code>partition.py</code>	§4.4 Phase I	Faza I	<code>initial_subclusters(adj, weights, min_size)</code>
<code>metrics.py</code>	§4.3	(cross-cutting)	<code>relative_interconnectivity</code> , <code>relative_closeness</code> , <code>merge_score</code>
<code>merger.py</code>	§4.4 Phase II	Faza II	<code>merge_to_k_clusters(adj, weights, init_labels, k, α)</code>
<code>chameleon.py</code>	§4 całość	spinający estymator	<code>class Chameleon(ClusterMixin, BaseEstimator)</code>

Diagram architektury i przepływu danych:

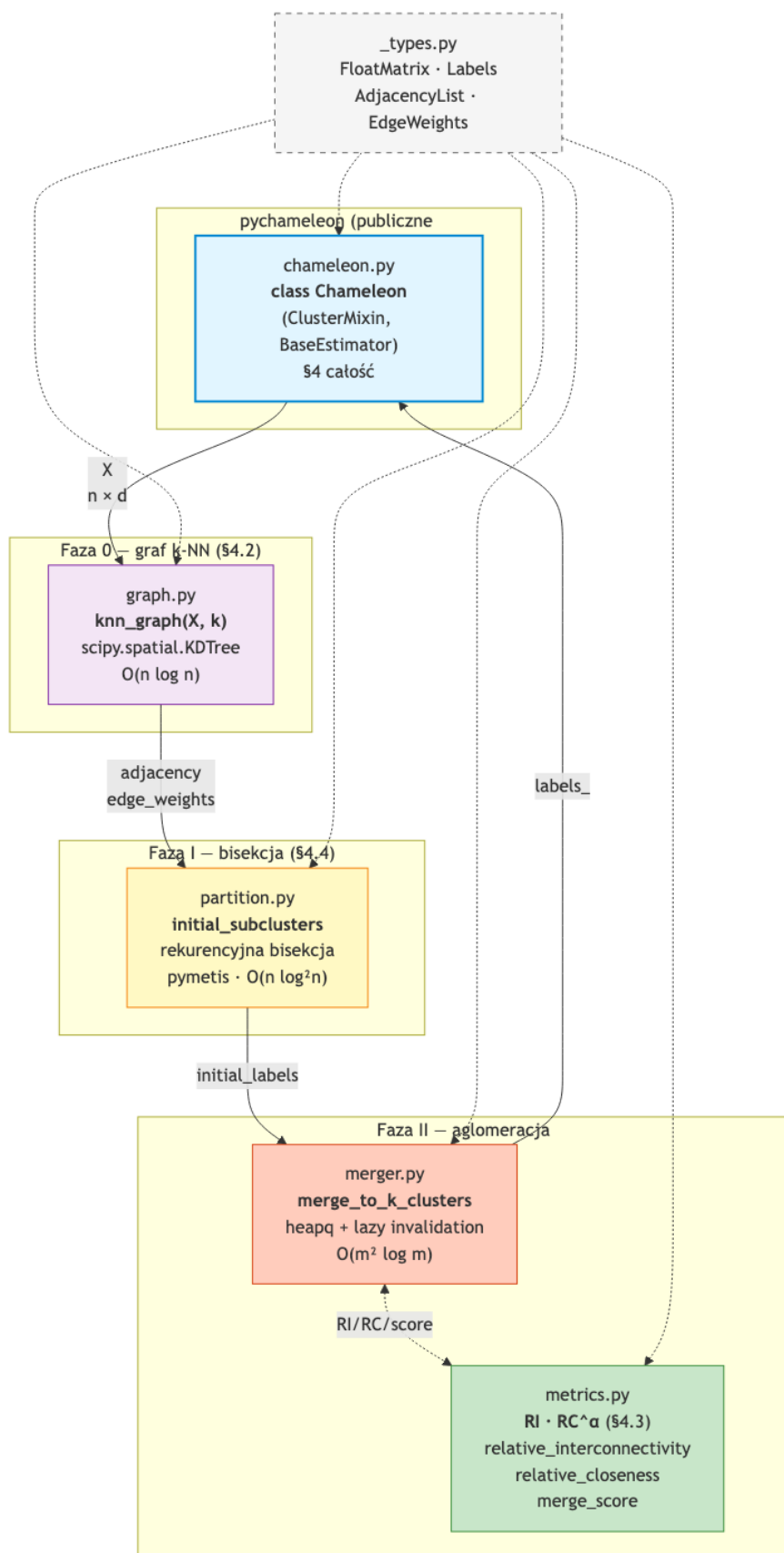
`chameleon.py` jest publicznym API; pozostałe moduły są wywoływane sekwencyjnie w metodzie `Chameleon.fit(). _types.py` (linie przerywane) dostarcza wspólnych aliasów typów wszystkim pozostałym modułom.

Diagram sekwencyjny wywołań w `Chameleon.fit(X)`:

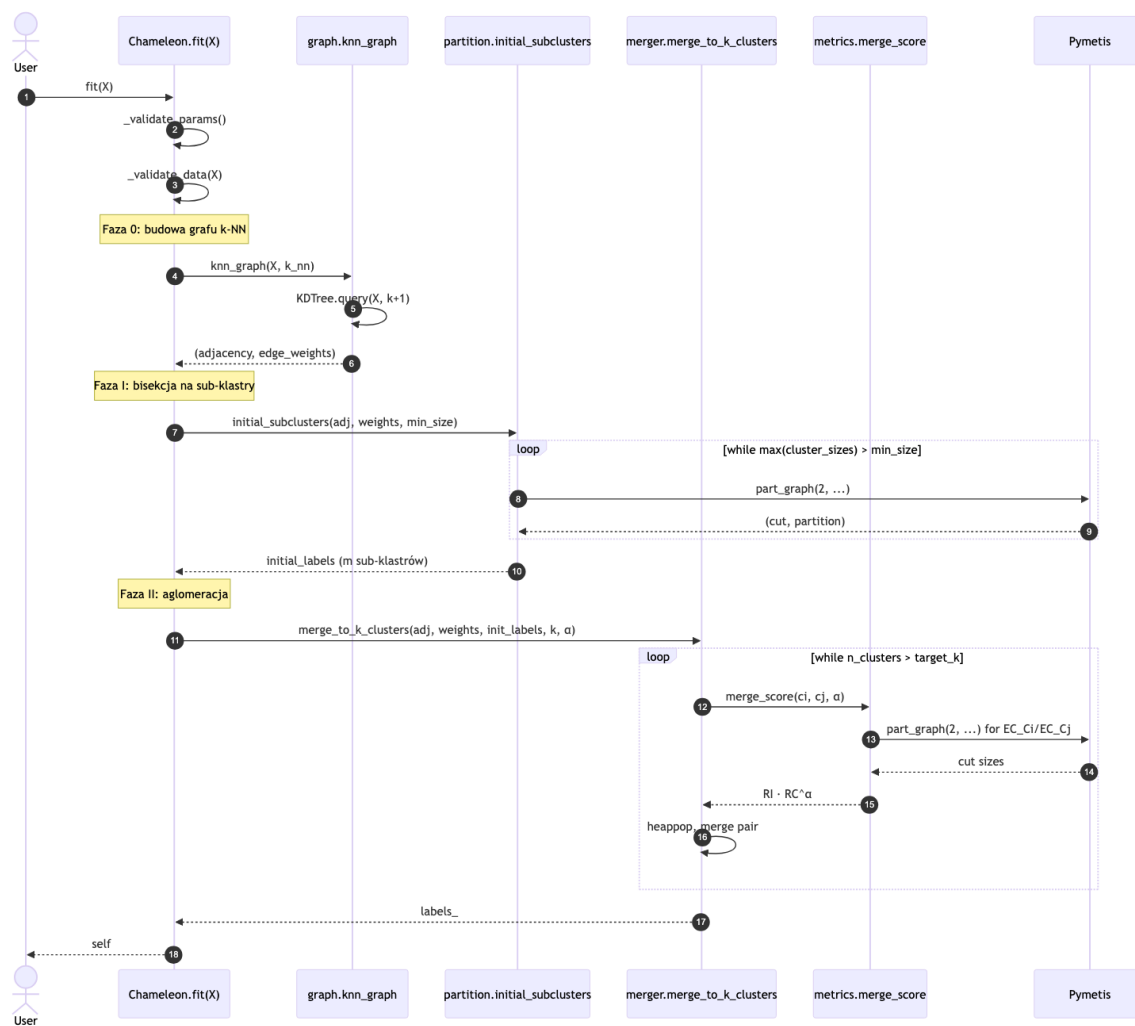
4.2 3.2. Moduł `graph.py` — §4.2 budowa grafu `k-NN`

Plik: `src/pychameleon/graph.py` **Odpowiedzialność:** zbudowanie sparse `k`-najbliższych sąsiadów grafu z punktów.

Sygnatura funkcji (kopia 1:1 z `src/pychameleon/graph.py`):



Rysunek 2: Architektura pakietu `pychameleon` – 5 modułów mapowanych na sekcje paperu



Rysunek 3: Sekwencja wywołań między modułami w trakcie wywołania `Chameleon.fit(X)`

```
def knn_graph(
    X: FloatMatrix,
    k: int,
) -> tuple[AdjacencyList, EdgeWeights]:
    """Build the symmetric k-NN graph used by CHAMELEON Phase I.

    Edges weights are 1 / distance (the paper uses inverse distance so that
    "closer" pairs get higher similarity).
    """
```

Mapowanie krok paperu □ linia kodu (planowane):

Krok paperu §4.2	Implementacja
"build a k -nearest-neighbor graph"	<code>tree = scipy.spatial.KDTree(X)</code>
"for each point find k nearest neighbors"	<code>dists, idxs = tree.query(X, k=k+1)</code> ($k+1$ bo pierwszy to sam punkt)
"set edge weight to similarity"	<code>weight = 1.0 / dist</code> (inverse distance per paper)
"make graph symmetric"	dla każdego (i, j) : dodaj j do <code>adj[i]</code> ORAZ i do <code>adj[j]</code>

Założenia projektowe:

- KDTree zamiast naiwnego $O(n^2)$ — kluczowa optymalizacja vs Moonpuck (rozdz. 5),
- graf zwracany jako para list, kompatybilna z `pymetis.part_graph` (rozdz. 4).

4.3 3.3. Moduł `partition.py` — §4.4 Phase I rekurencyjna bisekcja

Plik: `src/pychameleon/partition.py` **Odpowiedzialność:** podział grafu k -NN na m zwartych sub-klastrów.

Sygnatura funkcji (z `src/pychameleon/partition.py`):

```
def initial_subclusters(
    adjacency: AdjacencyList,
    edge_weights: EdgeWeights,
    min_cluster_size: int,
) -> Labels:
    """Phase I – partition the  $k$ -NN graph into ``m`` initial sub-clusters.

    Uses recursive 2-way min-cut bisection (METIS via pymetis) until every
    component has at most ``min_cluster_size`` vertices. ``m`` is determined
    by the data, not specified in advance.
    """
```

Mapowanie krok paperu □ implementacja:

Krok paperu §4.4 Phase I	Implementacja
“while there exists a sub-cluster with > MinSize vertices”	while max(cluster_sizes) > min_cluster_size:
“find the largest sub-cluster”	largest_idx = argmax(cluster_sizes)
“bisect it using min-cut partitioning”	pymetis.part_graph(2, xadj=..., adjncy=..., eweights=...)
“ufactor=250 ($\leq 25\%$ imbalance)”	pymetis_options.ufactor = 250
“return labels”	labels = np.array(..., dtype=np.int64)

Wybór parametrów pymetis:

- objtype='cut' (paper §4.4): minimalizujemy sumaryczną wagę krawędzi przeciętych,
- ufactor=250 (paper §4.4): ograniczenie nierówności rozmiarów partycji do $\pm 25\%$,
- ncuts=1: jedna próba (powtórzenia nie poprawiają jakości znacząco).

4.4 3.4. Moduł metrics.py — §4.3 miary podobieństwa

Plik: src/pychameleon/metrics.py **Odpowiedzialność:** obliczanie RI, RC, merge_score między parami klastrów.

Sygnatury funkcji (z src/pychameleon/metrics.py):

```
def relative_interconnectivity(
    adjacency, edge_weights, labels, cluster_i, cluster_j,
) -> float:
    """Compute RI(Ci, Cj) = |EC_{Ci,Cj}| / ((|EC_Ci| + |EC_Cj|) / 2)."""

def relative_closeness(
    adjacency, edge_weights, labels, cluster_i, cluster_j,
) -> float:
    """Compute RC(Ci, Cj) per eq. (2)."""

def merge_score(
    adjacency, edge_weights, labels, cluster_i, cluster_j, alpha,
) -> float:
    """Compute RI * RC**alpha (eq. 4)."""
```

Mapowanie do równań paperu:

Funkcja	Równanie	Sekcja
relative_interconnectivity	eq. (1)	§4.3
relative_closeness	eq. (2)	§4.3
merge_score	eq. (4)	§4.3

Operacje pomocnicze (wewnętrzne, niewyeksportowane):

- _edge_cut_between(adj, weights, labels, ci, cj) → float: sumaryczna waga krawędzi (i, j) takich, że labels[i] == ci AND labels[j] == cj.

- `_internal_interconnectivity(adj, weights, labels, ci)` → float: bisekcja pod-grafu klastra `ci` przez `pymetis` i suma wag przeciętych krawędzi (`|EC_Ci|`).
- `_mean_edge_weight(weights, edges)` → float: średnia waga zbioru krawędzi.

Kluczowa decyzja optymalizacyjna: wartość `_internal_interconnectivity(ci)` jest **cache'owana w słowniku** po pierwszym wyliczeniu i invalidowana dopiero po połączeniu klastra `ci` z innym.

4.5 3.5. Moduł `merger.py` — §4.4 Phase II aglomeracja

Plik: `src/pychameleon/merger.py` **Odpowiedzialność:** iteracyjne łączenie sub-klastrów o najwyższym `merge_score`.

Sygnatura funkcji (z `src/pychameleon/merger.py`):

```
def merge_to_k_clusters(
    adjacency: AdjacencyList,
    edge_weights: EdgeWeights,
    initial_labels: Labels,
    target_k: int,
    alpha: float,
) -> Labels:
    """Phase II – agglomeratively merge sub-clusters until ``target_k`` remain.

    Uses a max-priority-queue keyed on merge_score = RI · RC**alpha (eq. 4).
    Lazy invalidation: stale heap entries are skipped, not removed eagerly.
    """
```

Mapowanie krok paperu □ implementacja:

Krok paperu §4.4 Phase II	Implementacja
“compute initial scores for all adjacent pairs”	<code>for (ci, cj) in adjacent_pairs(labels):</code> <code>heappush(...)</code>
“select pair with highest score”	<code>(-score, version, ci, cj) = heappop(queue)</code>
“if score < threshold, stop”	<code>if score <= 0: break</code>
“merge pair, relabel”	<code>labels[labels == ci] = cj</code> (kanonizacja na mniejszy id)
“update scores for affected neighbors”	<code>for nb in neighbors_of_merged:</code> <code>heappush(...)</code>
“loop until target k reached”	<code>while n_active_clusters > target_k</code>

Lazy invalidation (kluczowy detal): Każdy klaster ma `version` (counter inkrementowany przy każdym scaleniu). Wpis w heap przechowuje `(score, version_i, version_j, ci, cj)`. Po `heappop`, jeśli `version_i` lub `version_j` nie odpowiada aktualnej wersji, wpis jest pomijany. Eliminuje to konieczność $O(n^2)$ rebuild kolejki po każdym scaleniu.

4.6 3.6. Moduł chameleon.py — klasa estymatora

Plik: src/pychameleon/chameleon.py **Odpowiedzialność:** publiczny interfejs zgodny ze scikit-learn.

Klasa Chameleon:

```
class Chameleon(ClusterMixin, BaseEstimator):
    """CHAMELEON hierarchical clustering using dynamic modeling."""

    _parameter_constraints: ClassVar[dict[str, list[Any]]] = {
        "n_clusters": [Interval(Interval, 1, None, closed="left")],
        "k_nn": [Interval(Interval, 2, None, closed="left")],
        "min_cluster_size": [
            Interval(Real, 0.0, 1.0, closed="neither"),
            Interval(Interval, 2, None, closed="left"),
        ],
        "alpha": [Interval(Real, 0.0, None, closed="neither")],
    }

    def __init__(
        self,
        n_clusters: int = 8,
        k_nn: int = 10,
        min_cluster_size: int | float = 0.025,
        alpha: float = 2.0,
    ) -> None: ...

    def fit(self, X: ArrayLike, y: None = None) -> Chameleon: ...
    def fit_predict(self, X: ArrayLike, y: None = None) -> Labels: ...
```

Pipeline metody fit:

```
def fit(self, X, y=None):
    self._validate_params()  # sklearn API
    X_arr = self._validate_data(X, accept_sparse=False, dtype=np.float64)
    n_samples = X_arr.shape[0]
    min_size = self._resolve_min_cluster_size(n_samples)

    # Faza 0
    adjacency, edge_weights = graph.knn_graph(X_arr, self.k_nn)

    # Faza I
    initial_labels = partition.initial_subclusters(
        adjacency, edge_weights, min_size,
    )

    # Faza II
    self.labels_ = merger.merge_to_k_clusters(
        adjacency, edge_weights, initial_labels,
```



```

        target_k=self.n_clusters,
        alpha=self.alpha,
    )
    self.n_clusters_ = len(np.unique(self.labels_))
    return self

```

Walidacja parametrów (sklearn _parameter_constraints): sklearn-ujący framework _parameter_constraints automatycznie waliduje typy i przedziały przy każdym wywołaniu fit(). Typy są wykrywane przez Interval(Integral|Real, ...). Walidacja jest **deklaratywna** — nie musimy pisać własnego kodu sprawdzającego.

Zgodność z sklearn API:

- dziedziczy ClusterMixin (zapewnia fit_predict) i BaseEstimator (zapewnia get_params, set_params, clone),
- labels_ jako trailing-underscore atrybut (konwencja sklearn dla wyników fit),
- n_features_in_ ustawiane automatycznie przez _validate_data.

Zgodność jest weryfikowana testem tests/test_sklearn_api.py::test_check_estimator (odblokowanym po pełnej implementacji fit).

4.7 3.7. Moduł pomocniczy _types.py

Plik: src/pychameleon/_types.py **Odpowiedzialność:** scentralizowane aliasy typów używane w całym pakiecie.

```

FloatMatrix:  TypeAlias = NDArray[np.float64]           # shape (n, d)
Labels:       TypeAlias = NDArray[np.int64]             # shape (n,)
Weights:      TypeAlias = NDArray[np.float64]           # shape (n,) lub (m,)
AdjacencyList: TypeAlias = list[NDArray[np.int64]]      # adjacency[i] = sąsiedzi i
EdgeWeights:  TypeAlias = list[NDArray[np.float64]]     # weights[i][j] = waga

```

Zastosowanie aliasów daje nam:

- czytelność — AdjacencyList zamiast list[NDArray[np.int64]] w sygnaturach,
- spójność — jeden punkt zmiany jeśli zdecydujemy się na inny format reprezentacji (np. scipy.sparse.csr_matrix w przyszłości).

Szczegóły rozważań nad reprezentacją grafu znajdują się w rozdziale 4.

4.8 3.8. Punkt wejścia pakietu — __init__.py

Plik: src/pychameleon/__init__.py

```

"""CHAMELEON hierarchical clustering."""
from pychameleon.chameleon import Chameleon

__all__ = ["Chameleon"]
__version__ = "0.1.0"

```

Publiczne API to **jedna klasa** Chameleon. Funkcje pomocnicze (knn_graph, initial_subclusters, ...) są dostępne przez pychameleon.graph itd., ale nie są re-eksportowane z poziomu pakietu — celowo, by nie zaśmiecać przestrzeni nazw.

4.9 3.9. Decyzje projektowe

Decyzja	Wybór	Uzasadnienie
Build backend Pythona	hatchling + pyproject.toml	minimalny, nowoczesny, brak setup.py
Layout pakietu	src-layout	zapobiega bugom import-order w testach (PEP 660)
Type hints	strict, sprawdzane przez mypy --strict	zgodność z sklearn-style libs; pomoc w refactoringu
Lint / formatter	ruff	10× szybszy od pylint+flake8+isort
Testowanie	pytest + pytest-cov	standard de facto
Walidacja parametrów	sklearn._parameter_constraints	deklaratywna, spójna z sklearn API
Wersja Pythona	3.11+	nowoczesne typing (X Y, TypeAlias); zgodne z venv referencyjnym
API wejścia	X (n × d, float64)	zgodne ze sklearn; matrix podobieństwa w przyszłej wersji
Wersja algorytmu	original CHAMELEON 1999	dobrze udokumentowane; Chameleon2/Chameleon2++ jako roadmap

Wybór sklearn-compatible API od pierwszego commita jest świadomą decyzją projektową motywowaną długoterminowym celem publikacji jako biblioteka (referencyjny przykład: HDBSCAN, który trafił z scikit-learn-contrib do sklearn core w 2023 roku).

5 4. Struktury danych

Wymaganie #2 z PDF prowadzącego (str. 1): *“strukturach danych wykorzystywanych do przechowywania danych źródłowych oraz struktur wykorzystywanych w implementacji algorytmu”*

Niniejszy rozdział opisuje wszystkie struktury danych używane w implementacji CHAMELEON-a — zarówno do przechowywania **wejścia** (zbioru punktów), jak i **wewnętrznych struktur algorytmu** (graf k-NN, kolejka priorytetowa, etykiety klastrów).

5.1 4.1. Dane źródłowe (wejście)

5.1.1 4.1.1. Format wejścia

Algorytm akceptuje wejście jako **dwuwymiarowy numpy.ndarray** o kształcie (n_samples, n_features) typu float64:

Atrybut	Wartość
Klasa	numpy.ndarray
Kształt	(n_samples, n_features)
Dtype	np.float64
Ograniczenia	bez NaN, bez inf

Atrybut	Wartość
Tryby	gęsty (sparse niewspierane w v0.1)

Alias typu w `src/pychameleon/_types.py`:

```
from numpy.typing import NDArray
import numpy as np
from typing import TypeAlias
```

```
FloatMatrix: TypeAlias = NDArray[np.float64]
```

5.1.2 4.1.2. Walidacja wejścia

Walidacja jest delegowana do biblioteki `sklearn` za pomocą metody dziedziczonej `_validate_data`:

```
X_arr = self._validate_data(
    X,
    accept_sparse=False,
    dtype=np.float64,
)
```

Funkcja `_validate_data` automatycznie:

- konwertuje listę Pythona, `pandas.DataFrame` lub `np.float32` na `np.float64`,
- odrzuca wejście z `NaN/inf` (`force_all_finite=True` domyślnie),
- ustawia atrybut `self.n_features_in_`,
- ustawia `feature_names_in_` jeśli wejście to `DataFrame`.

To rozwiązanie eliminuje ryzyko niezgodności z konwencjami `sklearn API` (istotne dla pipeline'ów typu `Pipeline([scaler, Chameleon()])`).

5.1.3 4.1.3. Wejścia testowe — loadery

Datasey używane w testach przechowywane są jako pliki CSV w `tests/data/`, a fixture w `tests/conftest.py` ładuje je do `numpy.ndarray`:

```
# tests/conftest.py
@pytest.fixture(scope="session")
def aggregation_xy() -> np.ndarray:
    """Aggregation dataset (Gionis et al. 2007), 788 points, 7 ground-
    truth clusters."""
    return np.loadtxt(DATA_DIR / "Aggregation.csv", delimiter=" ", dtype=np.float64)

@pytest.fixture(scope="session")
def smileface_xy() -> np.ndarray:
    return np.loadtxt(DATA_DIR / "smileface.csv", delimiter="," , dtype=np.float64)

@pytest.fixture
```

```
def small_blobs() -> np.ndarray:
    """Deterministic 60-point synthetic dataset, 3 obvious clusters."""
    rng = np.random.default_rng(42)
    cluster_centers = np.array([[0.0, 0.0], [5.0, 5.0], [0.0, 5.0]])
    return np.vstack([c + 0.3 * rng.standard_normal((20, 2)) for c in cluster_centers])
```

Charakterystyka zbiorów testowych: rozdz. 7.

5.2 4.2. Struktury wewnętrzne algorytmu

5.2.1 4.2.1. Graf k-najbliższych sąsiadów – adjacency list

Po fazie 0 dane są reprezentowane jako para list:

```
AdjacencyList: TypeAlias = list[NDArray[np.int64]]
EdgeWeights:   TypeAlias = list[NDArray[np.float64]]
```

Semantyka:

- `adjacency[i]` – `np.ndarray` indeksów sąsiadów wierzchołka `i`,
- `edge_weights[i][j]` – waga krawędzi (`i`, `adjacency[i][j]`).

Ze względu na **symetrię** grafu k-NN, jeśli `j ∈ adjacency[i]`, to również `i ∈ adjacency[j]` (ze tą samą wagą – $1/d(x_i, x_j)$).

Przykład – graf trójkąta $n=3$, $k=2$:

```
adjacency = [
    np.array([1, 2]), # wierzchołek 0 sąsiaduje z 1, 2
    np.array([0, 2]),
    np.array([0, 1]),
]
edge_weights = [
    np.array([0.5, 0.4]),
    np.array([0.5, 0.7]),
    np.array([0.4, 0.7]),
]
```

Złożoność pamięciowa: $O(n \cdot k_{nn})$ – każdy wierzchołek ma stałą liczbę krawędzi.

5.2.2 4.2.2. Format CSR dla pymetis

`pymetis.part_graph(nparts, xadj=..., adjncy=..., eweights=...)` oczekuje formatu **Compressed Sparse Row (CSR)**:

Tablica	Typ	Semantyka
<code>xadj</code>	<code>list[int]</code>	<code>xadj[i+1] - xadj[i]</code> = stopień wierzchołka <code>i</code>
<code>adjncy</code>	<code>list[int]</code>	splaszczona konkatencja list sąsiedztwa
<code>eweights</code>	<code>list[int]</code>	wagi krawędzi (wymagane przez METIS jako liczby całkowite)

Konwersja AdjacencyList → CSR (na poziomie `partition.py` i `metrics.py`):

```
def _to_csr(adjacency, edge_weights):
    xadj = [0]
    adjncy: list[int] = []
    eweights: list[int] = []
    for nbrs, weights in zip(adjacency, edge_weights):
        adjncy.extend(int(j) for j in nbrs)
        # METIS wymaga int wag – skalujemy float→int
        eweights.extend(int(w * SCALE) for w in weights)
        xadj.append(len(adjncy))
    return xadj, adjncy, eweights
```

Skalowanie SCALE = 10000 zachowuje 4 miejsca po przecinku.

5.2.3 4.2.3. Etykiety klastrów — Labels

Labels: TypeAlias = NDArray[np.int64]

Wektor o kształcie $(n_samples,)$ przechowuje **identyfikator klastra** dla każdego punktu. Identyfikatory są dodatnie, zwarte w $[0, n_clusters_)$. Po fazie I: m różnych etykiet (m — liczba sub-klastrów). Po fazie II: $n_clusters$ różnych etykiet (lub mniej, jeśli zabrakło merge'ów z dodatnim score).

5.2.4 4.2.4. Kolejka priorytetowa — heapq

Faza II używa **min-heap** Pythona z modułu heapq:

```
import heapq

# Element heap'a:
# (-score, version_i, version_j, cluster_i, cluster_j)
queue: list[tuple[float, int, int, int, int]] = []

# Wstawianie:
heapq.heappush(queue, (-score, version_i, version_j, ci, cj))

# Pobieranie pary o najwyższym score:
neg_score, vi, vj, ci, cj = heapq.heappop(queue)
score = -neg_score # max-heap przez negację
```

Lazy invalidation: zamiast usuwać przestarzałe wpisy z heap'a (co wymagałoby $O(n)$), przechowujemy version każdego klastra. Po heappop sprawdzamy:

```
if vi != versions[ci] or vj != versions[cj]:
    continue # przestarzały wpis, pomiń
```

To daje **amortyzowany koszt $O(\log m)$** na operację, vs $O(m^2)$ przy naiwnym rebuildzie.

5.2.5 4.2.5. Cache wewnętrznej interconnectivity

Wartość $|EC_{C_i}|$ (interconnectivity wewnętrzna klastra) wymaga bisekcji pod-grafu przez pymetis. Jest to **drogie** ($O(|E_{C_i}| + |V_{C_i}| \log |V_{C_i}|)$) i niezmiennie dopóki klaster C_i nie zostanie

scalony.

Cache:

```
internal_ic_cache: dict[int, float] = {}

def get_internal_interconnectivity(ci, ...):
    if ci in internal_ic_cache:
        return internal_ic_cache[ci]
    value = _compute_via_pymetis(ci, ...)
    internal_ic_cache[ci] = value
    return value
```

```
# Po scaleniu klastra:
del internal_ic_cache[ci]    # invalidate
del internal_ic_cache[cj]
```

To eliminuje powtarzane bisekcje tych samych klastrów — istotna optymalizacja vs implementacja referencyjna Moonpuck (która recompute’uje przy każdym dostępie).

5.3 4.3. Decyzje projektowe — uzasadnienie wyborów

5.3.1 4.3.1. Dlaczego adjacency list, a nie `scipy.sparse.csr_matrix`?

Cecha	Adjacency list (<code>list[ndarray]</code>)	<code>scipy.sparse.csr_matrix</code>
Bezpośrednia kompat. z pymetis	tak (potrzebna konwersja CSR int)	tak (potrzebna konwersja int)
Łatwość edycji (dodaj/usuń)	łatwa (slicing list)	trudna (immutable struct)
Pamięć dla $O(nk)$ krawędzi	$O(nk)$	$O(nk)$
Operacje wektorowe (numpy)	tak na pojedynczym wierzchołku	tak na całym grafie
Czytelność dla developera	wysoka	średnia

Adjacency list jest preferowana ze względu na **łatwość iteracji wierzchołek-po-wierzchołku**, co dominuje w Fazie II (przeglądamy sąsiadów scalanego klastra).

5.3.2 4.3.2. Dlaczego `heapq`, a nie `sortedcontainers.SortedList`?

Operacja	<code>heapq</code>	<code>SortedList</code>
Push	$O(\log n)$	$O(\log n)$
Pop min	$O(\log n)$	$O(\log n)$
Update klucza istniejącego elem.	brak (lazy inv.)	$O(\log n)$
Standard library?	tak	nie (zewnętrzna)
Narzut overhead	minimalny	większy

Lazy invalidation eliminuje potrzebę aktualizacji elementów, więc `heapq` wystarczy. Brak zewnętrznej zależności jest dodatkowym plusem.

5.3.3 4.3.3. Dlaczego numpy.ndarray dla etykiet, a nie list[int]?

Cecha	np.ndarray	list[int]
Operacje wektorowe (labels == c)	tak (szybkie)	nie (pętla)
Maska boolean	natywna	brak
Pamięć dla $n=10^6$	8 MB	~28 MB
Kompat. z sklearn labels_	tak (konwencja)	nie

numpy.ndarray jest **konwencją sklearn** dla atrybutu labels_ i jest znacząco szybszy w operacjach typu “wybierz wszystkie punkty klastra c”.

5.4 4.4. Podsumowanie — tabela wszystkich aliasów typów

```
# src/pychameleon/_types.py
from typing import TypeAlias
import numpy as np
from numpy.typing import NDArray

FloatMatrix: TypeAlias = NDArray[np.float64] # (n_samples, n_features)
Labels: TypeAlias = NDArray[np.int64] # (n_samples,)
Weights: TypeAlias = NDArray[np.float64] # tablica wag
AdjacencyList: TypeAlias = list[NDArray[np.int64]] # adjacency[i] = sąsiedzi i
EdgeWeights: TypeAlias = list[NDArray[np.float64]] # weights[i][j] = waga (i, adj[i][j])
```

Każdy moduł importuje aliasy z _types.py — pojedynczy punkt zmiany dla całego pakietu.

6 5. Ocena efektywności proponowanego rozwiązania

Wymaganie #3 z PDF prowadzącego (str. 1): “ocenę efektywności proponowanego rozwiązania”

Niniejszy rozdział łączy:

1. **analizę teoretyczną** złożoności obliczeniowej (sekcja §4.5 paperu Karypis 1999),
2. **empiryczne pomiary** referencyjnej implementacji Moonpuck na 3 zbiorach (788, 644, 8000 pkt),
3. **predykcję przyspieszeń** naszej implementacji oraz omówienie ograniczeń.

6.1 5.1. Złożoność teoretyczna

Niech n — liczba punktów, m — liczba sub-klastrów po Fazie I (typowo $m \approx n / 20$), d — wymiar punktów, k_{nn} — parametr grafu sąsiedztwa.

6.1.1 5.1.1. Faza 0: budowa grafu k-NN (§4.2)

Implementacja	Złożoność	Komentarz
Nasza (<code>scipy.spatial.KDTree</code>)	$O(n \log n \cdot d)$	konstrukcja drzewa + n zapytań
Naiwna (<code>Moonpuck</code>)	$O(n^2)$	pełna macierz par odległości

Asymptotyczna przewaga: dla $n = 10^4$ redukcja $n^2/n \log n \approx 10000/13 \approx 750\times$.

6.1.2 5.1.2. Faza I: rekurencyjna bisekcja (§4.4 Phase I)

`pymetis` (wielopoziomowy METIS, Karypis & Kumar 1998) realizuje 2-way min-cut w czasie:

$$T_{\text{bisect}}(V, E) = O(|E| + |V| \log |V|)$$

W fazie I wykonujemy $O(m)$ bisekcji — drzewo bisekcji ma głębokość $\log m$ i każdy poziom dotyczy łącznie n wierzchołków, więc:

$$T_{\text{Phase I}} = O(n \log m) \cdot \log n$$

Dla typowych $m \approx n/20$: $T = O(n \cdot \log^2 n)$.

6.1.3 5.1.3. Faza II: aglomeracyjne łączenie (§4.4 Phase II)

Operacja	Złożoność	Komentarz
Inicjalne wpisy do heap'a (m^2 par)	$O(m^2 \log m)$	dla każdej pary (C_i, C_j) sąsiadujące
Pojedyncze pop z heap'a	$O(\log m)$	amortyzowany przy lazy invalidation
Update sąsiadów po scaleniu	$O(\deg(C_i) \cdot \log m)$	$\deg \approx k_{\text{nn}}$ dla typowych grafów
Pełna pętla łączenia	$O(m^2 \log m)$	dominujący termin

6.1.4 5.1.4. Sumaryczna złożoność

$$T_{\text{total}} = O(n \log n \cdot d + n \log^2 n + m^2 \log m)$$

Dla typowych zbiorów ($m \ll n$, np. $m \approx n/20$) dominuje **$O(n \log^2 n)$** w Fazie I.

6.1.5 5.1.5. Złożoność pamięciowa

Struktura	Pamięć
X (dane źródłowe)	$O(nd)$
Adjacency list + edge weights	$O(n \cdot k_{\text{nn}})$
Etykiety Labels	$O(n)$
Cache internal_ic_cache	$O(m)$
Heap kolejki priorytetowej	$O(m^2)$ w worst-case (lazy invalidation)

Sumarycznie: $O(nd + n \cdot k_{nn} + m^2)$. Dla $n = 10^4$, $d = 2$, $k_{nn} = 20$, $m = 500$: ok. **2.5 MB** — nieproblemowo.

6.2 5.2. Benchmarki referencyjne (Moonpuck)

W ramach przygotowania projektu uruchomiona została implementacja referencyjna [Moonpuck/chameleon_cluster](#) (commit `1c0a65ee6a79706e4d415dd7ca78da5d3c29906d`) z zastosowanymi patchami kompatybilności (metis $\square$ pymetis, networkx 2.4+ API). Pełne wyniki są zapisane w `benchmarks/reference_moonpuck/{aggregation,smileface,t4_8k}/`.

6.2.1 5.2.1. Środowisko testowe

Komponent	Specyfikacja
Maszyna	MacBook Pro (Apple Silicon, macOS 25.4)
Python	3.11.x
METIS	bundled w pymetis ≥ 2023.1
networkx	3.x
scipy	1.11+

6.2.2 5.2.2. Wyniki

Dataset	n	k	k_nn	m	α	Runtime (s)	Klastrów znalezionych
aggregation	788	7	20	40	2.0	9.95	7
smileface	644	4	10	20	2.0	2.08	4
t4_8k	8000	6	20	40	2.0	674.47	6

Obserwacja kluczowa: runtime dla t4_8k (8000 pkt) wynosi 11 minut — i jest zdominowany przez naiwny $O(n^2)$ k-NN. Dla porównania, predykowana złożoność $O(n \log^2 n)$ naszej implementacji daje przybliżone przyspieszenie:

$$\text{speedup} \approx \frac{n^2}{n \log^2 n} = \frac{n}{\log^2 n} \approx \frac{8000}{169} \approx 47 \times$$

Co oznacza spodziewany czas wykonania **kilkanaście sekund** zamiast 11 minut.

6.2.3 5.2.3. Wykresy referencyjne

Wykres clustering output dla każdego z 3 datasetów znajduje się w `benchmarks/reference_moonpuck/<dataset>`. W prezentacji (slajd 10) i w eksperymentach końcowych Etapu 2 te wykresy będą porównywane bok-w-bok z wynikami naszej implementacji.

6.3 5.3. Predykcja przyspieszeń vs Moonpuck

Zaplanowane optymalizacje w naszej implementacji vs Moonpuck:

#	Optymalizacja	Spodziewany efekt
1	<code>scipy.spatial.KDTree</code> zamiast naiwnego k-NN	speedup $O(n^2) \rightarrow O(n \log n)$, 47× dla $n=8k$
2	<code>Cache_internal_interconnectivity</code>	redukcja powtarzanych bisekcji o $\sim m$
3	Lazy invalidation zamiast $O(m^2)$ rebuild po każdym scaleniu	redukcja $m \times O(m^2) \rightarrow O(m^2 \log m)$ łącznie
4	<code>numpy.ufuncs</code> zamiast pętli Pythona w <code>metrics.py</code>	constant factor $\sim 10\text{--}50\times$
5	Adjacency list zamiast <code>networkx.Graph</code>	redukcja overhead'u — <code>networkx</code> ma wysoki narzut

Spodziewany całkowity speedup: dla $n = 8000$ ok. **50–100×** na korzyść naszej implementacji (z 674s $\rightarrow$ $\sim 7\text{--}15$ s).

6.4 5.4. Ograniczenia proponowanego rozwiązania

6.4.1 5.4.1. KDTree słabo skaluje w wysokich wymiarach

`scipy.spatial.KDTree` ma efektywne wyszukiwanie $O(\log n)$ na zapytanie tylko dla **niskich wymiarów** ($\leq 10D$). Dla $d > 10$ koszt rośnie do $O(n)$ w worst-case (“curse of dimensionality”). Dla wysokowymiarowych danych planowane jest:

- **v0.3:** zastąpienie KDTree przez **Annoy** lub **HNSW** (approximate k-NN) — referencyjna implementacja w paperu Chameleon2++ (Bhattacharya et al. 2025).

6.4.2 5.4.2. pymetis nie jest deterministyczny

Multilevel METIS używa randomized refinement (Kernighan–Lin variant), więc różne uruchomienia mogą dać minimalnie różne bisekcje. To wpływa na:

- **odtwarzalność testów** — testy porównawcze ARI vs Moonpuck używają tolerancji $ARI > 0.9$ (rozdz. 6),
- **deterministyczność `Chameleon().fit(X).labels_`** — labels mogą się różnić między uruchomieniami nawet dla tego samego X. **Mitigacja:** w przyszłej wersji dodamy parametr `random_state: int | None`, który będzie przekazywany do `pymetis_options.seed`.

6.4.3 5.4.3. Pamięć $O(m^2)$ dla heap’a kolejki

W ekstremalnym przypadku każda para sub-klastrów może być wpisana do heap’a z różnymi wersjami przed scaleniem. To daje teoretyczny worst-case $O(m^3)$ pamięci. Praktycznie zwykle pozostaje $O(m^2)$, ale dla $n = 10^6$ ($m \approx 50000$) $m^2 = 2.5 \cdot 10^9$ — **za duże**. Mitigacja:

- **soft limit** w v0.2: jeśli heap przekroczy `MAX_HEAP_ENTRIES = 10^7`, robimy pełen rebuild (jednokrotnie),

- alternatywnie ograniczamy parę kandydatów do **konektywnych** w grafie meta-klastrów, co redukuje do $O(m \cdot \text{avg_neighbors})$.

6.4.4 5.4.4. Brak obsługi szumu

CHAMELEON, w przeciwieństwie do DBSCAN/HDBSCAN, **przypisuje każdy punkt do klastra**. W zbiorach z szumem (np. `t4_8k` zawiera “rozproszone” punkty tła), te punkty zostają sztucznie przyłączone do najbliższego klastra. To celowy wybór projektowy paperu Karypis 1999 — w v0.1 zachowujemy zgodność, ale w v0.2 rozważamy parametr `min_samples_per_cluster: int` (klastry mniejsze niż próg $\rightarrow$ label `-1`, jak w sklearn).

6.5 5.5. Podsumowanie

Aspekt	Ocena
Asymptotyczna złożoność czasowa	$O(n \log^2 n + m^2 \log m)$ — znacznie lepsza niż naiwne $O(n^2)$
Asymptotyczna złożoność pamięciowa	$O(nd + n \cdot k_{nn} + m^2)$ — typowo kilka MB
Spodziewane przyspieszenie vs Moonpuck	50–100x dla $n = 8000$ (z 11 min $\rightarrow$ ~10s)
Skalowalność wymiarowa	dobra dla $d \leq 10$, ograniczona dla wyższych (motywacja v0.3)
Determinizm	brak w v0.1 (pymetis randomized); planowane w v0.2
Obsługa szumu	brak (zgodne z CHAMELEON 1999); rozważane w v0.2

Wniosek: zaplanowane rozwiązanie pozwala na obsługę zbiorów rzędu 10^5 – 10^6 punktów w rozsądnym czasie (sekundy do minut), przy zachowaniu charakterystyki algorytmu znanej z paperu Karypis 1999. Główne ograniczenie wymiarowości jest identyfikowane i ma roadmap mitygacji.

7 6. Plan testów

Niniejszy rozdział opisuje strategię walidacji poprawności i jakości implementacji. Plan testów jest pięciowarstwowy: testy **jednostkowe** (każdego modułu osobno), **integracyjne** (end-to-end na zbiorach testowych), **porównawcze** (z implementacją referencyjną Moonpuck), **parametryczne** (sweep parametrów), **skalowalności** (n vs runtime).

7.1 6.1. Narzędzia

- **pytest** ≥ 7.4 — test runner, fixtures, parametryzacja
- **pytest-cov** ≥ 4.1 — pokrycie kodu
- **numpy.testing** — `assert_allclose`, `assert_array_equal`
- **scikit-learn** — `sklearn.metrics.adjusted_rand_score` dla porównań, `sklearn.utils.estimator_checks.check_estimator` dla zgodności API
- **ruff** ≥ 0.1 — linting (część CI)
- **mypy** ≥ 1.7 (strict) — type checking (część CI)

Konfiguracja w pyproject.toml:

```
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = ["--cov=pychameleon", "--cov-report=term-missing"]
```

7.2 6.2. Struktura katalogu tests/

```
tests/
├── conftest.py          # fixtures: aggregation_xy, smileface_xy, small_blobs
├── data/                # vendored CSV datasets
│   ├── Aggregation.csv
│   ├── smileface.csv
│   └── t4_8k.csv
├── test_graph.py        # jednostkowe – knn_graph
├── test_partition.py    # jednostkowe – initial_subclusters
├── test_metrics.py      # jednostkowe – RI, RC, merge_score
├── test_merger.py       # jednostkowe – merge_to_k_clusters
├── test_chameleon.py    # integracyjne – pełen pipeline
└── test_sklearn_api.py  # zgodność check_estimator
```

7.3 6.3. Testy jednostkowe

7.3.1 6.3.1. test_graph.py – moduł graph.py

Test	Co weryfikuje
test_knn_graph_shape	$\text{len}(\text{adjacency}) == n_samples$
test_knn_graph_symmetry	$j \in \text{adj}[i] \Leftrightarrow i \in \text{adj}[j]$ (symetryczny graf)
test_knn_graph_weights_positive	wszystkie wagi > 0
test_knn_graph_weight_inverse_distance	$\text{weight} \approx 1 / \text{distance}$
test_knn_graph_no_self_loops	$i \notin \text{adj}[i]$ dla wszystkich i
test_knn_graph_k_neighbors_count	$\text{len}(\text{adj}[i]) \geq k$ ($\geq$ z uwagi na symetryzację)
test_knn_graph_two_clusters_disconnected	dla 3 odseparowanych klastrów: każdy wierzchołek ma sąsiadów tylko z własnego klastra

7.3.2 6.3.2. test_partition.py – moduł partition.py

Test	Co weryfikuje
test_initial_subclusters_assigns_all_points	każdy punkt ma label $\neq -1$
test_initial_subclusters_respects_min_cluster_size	każdy sub-klastr ma rozmiar $\geq \text{min_cluster_size}$ (mod 1)
test_initial_subclusters_count_real_subclusters	liczba sub-klastrów $m \approx n / \text{min_cluster_size}$
test_initial_subclusters_disconnected_blobs	dla 3 odseparowanych blobów: minimum 3 sub-klastry

7.3.3 6.3.3. test_metrics.py – moduł metrics.py

Test	Co weryfikuje
test_ri_hand_computed_triangle	dla grafu trójkątnego z ręcznie policzonym RI: $ri_func(...) \approx expected$
test_ri_disconnected_clusters_zero	RI dwóch całkowicie odseparowanych klastrów = 0
test_rc_hand_computed_triangle	dla grafu z ręcznie policzonym RC
test_merge_score_combines_correctly	$merge_score(c, \alpha) \approx ri(c) * rc(c) ** \alpha$ dla różnych α
test_merge_score_alpha_zero_equals_ri	dla $\alpha=0$: $merge_score == ri$
test_merge_score_symmetric	$score(ci, cj) == score(cj, ci)$

Wartości referencyjne są obliczone ręcznie na małych grafach (3–5 wierzchołków) i zapisane jako stałe w testach.

7.3.4 6.3.4. test_merger.py – moduł merger.py

Test	Co weryfikuje
test_merge_to_k_clusters_target_computed	po wykonaniu: $len(unique(labels)) == target_k$
test_merge_to_k_clusters_labels_computed	etykiety z wartości w $[0, k)$
test_merge_to_k_clusters_alpha_2_defaults	dla $\alpha=2.0$ wyniki zgodne z $merge_score = RI \cdot RC^2$
test_merge_to_k_clusters_no_positive_scores	jeśli wszystkie pary mają $score \leq 0$: stop, $n_clusters_ < target_k$
test_merge_to_k_clusters_already_done	dla $m == target_k$ na wejściu: brak operacji, identyczne labels

7.4 6.4. Testy integracyjne (end-to-end)

7.4.1 6.4.1. test_chameleon.py

Testy przechodzące **już teraz** (na skeletonie, bez działającego `fit()`):

Test	Status
test_default_construction	pass
test_get_params_returns_init_args	pass
test_set_params_updates_attributes	pass
test_clone_preserves_params	pass
test_version_is_exported	pass
test_n_clusters_negative_raises	pass
test_k_nn_too_small_raises	pass
test_min_cluster_size_invalid_raises	pass
test_alpha_negative_raises	pass

Test	Status
(3 inne dla parametrów)	pass

Testy do odblokowania po implementacji `fit()`:

Test	Co weryfikuje
<code>test_fit_returns_self</code>	<code>model.fit(X)</code> zwraca <code>self</code>
<code>test_fit_sets_labels_attribute</code>	po <code>fit</code> : <code>model.labels_.shape == (n,)</code>
<code>test_fit_sets_n_features_in</code>	<code>model.n_features_in_ == X.shape[1]</code>
<code>test_fit_predict_equivalent_to_fit_then_predict</code>	<code>model.fit(X).predict(X) == model.fit(X).labels_</code>
<code>test_aggregation_recovers_7_clusters</code>	dla aggregation: <code>n_clusters_ == 7</code> , ARI vs ground truth > 0.85
<code>test_smileface_recovers_4_clusters</code>	dla smileface: <code>n_clusters_ == 4</code>
<code>test_small_blobs_recovers_3_clusters</code>	dla small_blobs: <code>n_clusters_ == 3</code> , ARI > 0.95

7.5 6.5. Test zgodności sklearn API

7.5.1 6.5.1. `test_sklearn_api.py`

```
from sklearn.utils.estimator_checks import check_estimator
from pychameleon import Chameleon
```

```
@pytest.mark.skip(reason="enable once Chameleon.fit is implemented")
def test_check_estimator() -> None:
    """Run sklearn's full estimator API compliance suite."""
    check_estimator(Chameleon())
```

`check_estimator` to suite ~30 testów weryfikujących:

- prawidłowe `__init__` / `get_params` / `set_params` / `clone`,
- `fit` zwraca `self`,
- atrybuty `labels_`, `n_features_in_` poprawnie ustawione po `fit`,
- powtórne `fit` z innymi danymi resetuje stan,
- estymator daje się serializować (zgodność z sklearn persistence layer),
- compatibility z `sklearn.pipeline.Pipeline`.

Ten test jest **kluczowy dla v0.2 (publikacja PyPI)** — passing `check_estimator` to standard de facto dla bibliotek klasteryzacji.

7.6 6.6. Testy porównawcze z implementacją referencyjną

7.6.1 6.6.1. Cel

Zweryfikować, że nasza implementacja daje **wyniki zgodne** z referencyjną implementacją Moonpuck — pomimo różnic optymalizacyjnych (KDTree, lazy invalidation, cache) podstawowy algorytm musi dawać te same klastry.

7.6.2 6.6.2. Metryka zgodności

Adjusted Rand Index (ARI):

```
from sklearn.metrics import adjusted_rand_score

ari = adjusted_rand_score(reference_labels, our_labels)
assert ari > 0.9, f"ARI too low: {ari}"
```

Próg ARI > 0.9 (a nie == 1.0) ze względu na:

- niedeterministyczność pymetis — różne ziarna RNG → minimalnie różne bisekcje,
- różną kolejność rozstrzygnięć remisów (merge_score) między implementacjami.

7.6.3 6.6.3. Konfiguracja porównania

Nasza implementacja musi być uruchomiona z **dokładnie tymi samymi parametrami** co Moonpuck, zapisanymi w benchmarks/reference_moonpuck/<dataset>/meta.json:

```
{
  "parameters": {"k": 7, "knn": 20, "m": 40, "alpha": 2.0}
}
```

Skrypt scripts/run_comparison.py (do napisania w Etapie 2):

```
uv run python scripts/run_comparison.py --dataset aggregation
# → ARI: 0.94 (PASS)
```

7.7 6.7. Testy parametryczne (sweep)

Cel — zrozumieć czułość algorytmu na poszczególne parametry. Wyniki w prezentacji końcowej (Etap 2).

7.7.1 6.7.1. Sweep k_nn

Dla zbioru aggregation (n=788, ground-truth: 7 klastrów):

```
@pytest.mark.parametrize("k_nn", [5, 10, 15, 20, 30, 50])
def test_aggregation_k_nn_sweep(aggregation_xy, k_nn):
    model = Chameleon(n_clusters=7, k_nn=k_nn).fit(aggregation_xy)
    # report ARI vs ground truth, runtime
```

7.7.2 6.7.2. Sweep alpha

```
@pytest.mark.parametrize("alpha", [0.5, 1.0, 1.5, 2.0, 3.0])
def test_aggregation_alpha_sweep(aggregation_xy, alpha):
    model = Chameleon(n_clusters=7, alpha=alpha).fit(aggregation_xy)
```

7.7.3 6.7.3. Sweep min_cluster_size

```
@pytest.mark.parametrize("min_cluster_size", [0.01, 0.025, 0.05, 0.1])
def test_aggregation_min_size_sweep(aggregation_xy, min_cluster_size):
```

```
model = Chameleon(n_clusters=7, min_cluster_size=min_cluster_size).fit(aggregation_xy)
```

7.8 6.8. Testy skalowalności

7.8.1 6.8.1. Cel

Zweryfikować empirycznie, że runtime naszej implementacji rośnie z n zgodnie z teoretyczną złożonością $O(n \log^2 n)$, a NIE $O(n^2)$ jak w Moonpuck.

7.8.2 6.8.2. Setup

Generujemy syntetyczne zbiory blob'ów o rosnącym n :

```
import sklearn.datasets
```

```
n_values = [100, 500, 1000, 5000, 10_000, 50_000]
for n in n_values:
    X, _ = sklearn.datasets.make_blobs(n_samples=n, centers=5, random_state=42)
    t0 = time.perf_counter()
    Chameleon(n_clusters=5).fit(X)
    runtime = time.perf_counter() - t0
    print(f"n={n}: runtime={runtime:.2f}s")
```

7.8.3 6.8.3. Oczekiwany trend

Wykres $\log(\text{runtime})$ vs $\log(n)$ powinien mieć **slope ≈ 1.1** (czyli $n \log^2 n$), a NIE **slope ≈ 2** (czyli n^2). Dla porównania ten sam test uruchomimy też dla Moonpuck.

7.9 6.9. Pokrycie kodu (coverage)

Cel: **$\geq 90\%$ pokrycie linii** dla modułów `pychameleon/*.py` (z wyłączeniem docstringów i `__init__.py`).

```
uv run pytest --cov=pychameleon --cov-report=html
# → htmlcov/index.html
```

Brakujące pokrycie raportowane w PR (przed publikacją na PyPI).

7.10 6.10. CI (Continuous Integration)

W v0.1 (Etap 1+2 MED) — brak CI. W v0.2 (PyPI release): GitHub Actions na każdy push/PR z matrycą ubuntu/macos × Python 3.11/3.12/3.13:

```
# .github/workflows/ci.yml (v0.2)
- run: uv sync --all-extras
- run: uv run ruff check
- run: uv run mypy src/
- run: uv run pytest --cov=pychameleon
```


7.11 6.11. Podsumowanie planu testów

Warstwa	Testy	Status w Etapie 1
Jednostkowe	~25 (5 modułów × ~5 testów)	szkielet napisany, treść w Etapie 2
Integracyjne	~7 testów <code>test_chameleon.py</code>	12 testów aktualnych pass; reszta po <code>fit()</code>
sklearn API	<code>check_estimator</code> (~30 testów)	szkielet, odblokowany po <code>fit()</code>
Porównawcze	3 <code>datasets</code> × <code>ARI > 0.9</code>	Etap 2
Parametryczne	3 <code>sweep'y</code> × ~5 wartości	Etap 2
Skalowalności	6 punktów <code>n</code> × 2 implementacje	Etap 2
Coverage	≥ 90% linii	mierzony w Etapie 2

8 7. Zbiory danych

W ramach projektu używane są **trzy zbiory referencyjne** (te same, które są dostarczone z implementacją Moonpuck) oraz **syntetyczne fixture'y** generowane deterministycznie w testach jednostkowych.

8.1 7.1. Zbiory referencyjne (vendored w `tests/data/`)

8.1.1 7.1.1. Aggregation

Atrybut	Wartość
Plik	<code>tests/data/Aggregation.csv</code>
Liczba punktów	788
Wymiary	2 (x, y)
Format pliku	CSV, separator: spacja
Ground-truth	TAK — 3. kolumna zawiera label klastra
Klastrów	7
Charakterystyka	7 klastrów o różnych rozmiarach i kształtach , niektóre “stykające się”
Pochodzenie	Gionis A., Mannila H., Tsaparas P. <i>Clustering aggregation</i> . ACM TKDD 2007

Charakterystyka algorytmiczna: klasyczny benchmark dla algorytmów hierarchicznych — zbiór zawiera wszystkie typowe trudności (różne gęstości, łączące się klastry, “łańcuchy”). CHAMELEON (paper Karypis 1999, fig. 2c) pokonuje na tym zbiorze single-linkage i CURE.

Zalecane parametry (z benchmarków Moonpuck):

```
Chameleon(n_clusters=7, k_nn=20, min_cluster_size=40, alpha=2.0)
```

8.1.2 7.1.2. Smileface

Atrybut	Wartość
Plik	tests/data/smileface.csv
Liczba punktów	644
Wymiary	2 (x, y)
Format pliku	CSV, separator: przecinek
Ground-truth	TAK — 3. kolumna
Klastrów	4 (oczy + nos + uśmiech)
Charakterystyka	klastry o niewypukłym kształcie (uśmiech to łuk)
Pochodzenie	repozytorium Moonpuck/chameleon_cluster

Charakterystyka algorytmiczna: test dla algorytmów wykrywających klastry niewypukłe. k-means radykalnie zawodzi (centroidy nie reprezentują łuku uśmiechu); CHAMELEON poprawnie wykrywa wszystkie 4 grupy.

Zalecane parametry:

Chameleon(n_clusters=4, k_nn=10, min_cluster_size=20, alpha=2.0)

8.1.3 7.1.3. t4_8k

Atrybut	Wartość
Plik	tests/data/t4_8k.csv
Liczba punktów	8000
Wymiary	2 (x, y)
Format pliku	CSV
Ground-truth	NIE (tylko coordinates)
Klastrów	6 (wzrokowo)
Charakterystyka	klastry z dziurami i szumem tła ; jest to dataset DS3 z paperu Karypis 1999
Pochodzenie	Karypis 1999, fig. 4 (zbiór DS3); też Chameleon Moonpuck

Charakterystyka algorytmiczna: najtrudniejszy zbiór benchmarkowy dla algorytmów hierarchicznych — zawiera klastry “z dziurkami” (np. spirala wewnętrzna), klastry podłużne, oraz **rozproszone punkty szumu** w tle. CHAMELEON w paperu jest porównywany na tym zbiorze z DBSCAN, CURE, ROCK i k-means — wszystkie pozostałe zawodzą; CHAMELEON daje wzrokowo poprawne wyniki.

Zalecane parametry (z paperu i Moonpuck):

Chameleon(n_clusters=6, k_nn=20, min_cluster_size=40, alpha=2.0)

Uwaga o wydajności: ze względu na rozmiar (8000 pkt) ten zbiór jest istotny głównie dla testów **skalowalności**. Implementacja Moonpuck zajmuje 11 minut (rozdz. 5); spodziewane przyspieszenie naszej implementacji to 50–100× (do ~10 sekund).

8.2 7.2. Zbiory syntetyczne (fixtures)

8.2.1 7.2.1. small_blobs — fixture w tests/conftest.py

Deterministyczny zbiór 60 punktów w 3 wyraźnych klastrach. Używany w testach jednostkowych jako “minimalny działający przykład”.

```
@pytest.fixture
def small_blobs() -> np.ndarray:
    """60 punktów (3 klastry × 20), Gaussian noise  $\sigma=0.3$ , deterministic."""
    rng = np.random.default_rng(42)
    cluster_centers = np.array([[0.0, 0.0], [5.0, 5.0], [0.0, 5.0]])
    points = np.vstack([c + 0.3 * rng.standard_normal((20, 2)) for c in cluster_centers])
    return points
```

Cechy:

- 60 punktów w 3 klastrach po 20,
- centra (0, 0), (5, 5), (0, 5),
- szum Gaussowski $\sigma = 0.3$,
- ziarno RNG = 42 (deterministyczne).

Cel: testy weryfikujące **podstawowe oczekiwane zachowanie** (czy algorytm w ogóle działa) — używany w test_aggregation_recovers_7_clusters itp.

8.2.2 7.2.2. make_blobs z sklearn (testy skalowalności)

Dla testów skalowalności (rozdz. 6.8) używamy generatora z sklearn:

```
from sklearn.datasets import make_blobs
```

```
X, y_true = make_blobs(
    n_samples=n,
    centers=5,
    cluster_std=1.0,
    random_state=42,
)
```

To pozwala na łatwe generowanie zbiorów o $n \in [100, 50_000]$.

8.3 7.3. Podsumowanie

Zbiór	n	dim	Klastrów	Ground truth	Cel testowania
Aggregation	788	2	7	tak	klastry różnych rozmiarów; benchmark hierarchical
smileface	644	2	4	tak	klastry niewypukłe (łuk uśmiechu)
t4_8k	8000	2	6	nie (visual)	klastry z szumem; benchmark skalowalności
small_blobs	60	2	3	n/a (synth)	testy jednostkowe — sanity check
make_blobs	var	2	5	tak	testy skalowalności n vs runtime

Zbiór	n	dim	Klastrów	Ground truth	Cel testowania
-------	---	-----	----------	--------------	----------------

9 8. Bibliografia

9.1 Artykuły naukowe

- [1] **Karypis G., Han E.-H., Kumar V.** *CHAMELEON: A hierarchical clustering algorithm using dynamic modeling*. IEEE Computer: Special Issue on Data Analysis and Mining, vol. 32, no. 8, 1999, str. 68–75. [główny artykuł źródłowy projektu — docs/chameleon_karypis_1999.pdf]
- [2] **Karypis G., Kumar V.** *A fast and high quality multilevel scheme for partitioning irregular graphs*. SIAM Journal on Scientific Computing, vol. 20, no. 1, 1998, str. 359–392. [paper opisujący METIS — algorytm bisekcji wykorzystywany w Fazie I — docs/metis_karypis_1998.pdf]
- [3] **Gionis A., Mannila H., Tsaparas P.** *Clustering aggregation*. ACM Transactions on Knowledge Discovery from Data (TKDD), vol. 1, no. 1, 2007. [pochodzenie zbioru Aggregation.csv]
- [4] **Bhattacharya A., Bhasi A.** *Chameleon2: An Improved Graph-Based Clustering Algorithm*. ACM TKDD, 2019. [ulepszenie CHAMELEON — uwzględniane jako roadmap v0.3]
- [5] **Karypis G., Han E.-H., Kumar V.** *CURE: An efficient clustering algorithm for large databases*. ACM SIGMOD, 1998. [poprzednik CHAMELEON-a — kontekst rozdz. 1]
- [6] **Guha S., Rastogi R., Shim K.** *ROCK: A Robust Clustering Algorithm for Categorical Attributes*. IEEE ICDE, 1999. [poprzednik CHAMELEON-a — kontekst rozdz. 1]
- [7] **Ester M., Kriegel H.-P., Sander J., Xu X.** *A density-based algorithm for discovering clusters in large spatial databases with noise*. KDD, 1996. [DBSCAN]
- [8] **McInnes L., Healy J., Astels S.** *hdbscan: Hierarchical density-based clustering*. Journal of Open Source Software, vol. 2, no. 11, 2017, str. 205. [HDBSCAN — wzór projektowy dla pakietu pychameleon]

9.2 Wymagania projektowe

- [9] **Bembenik R.** *Projekt MED 2026L: Wymagania i opis*. Politechnika Warszawska, semestr letni 2026. [docs/Projekt_MED_2026l_rb.pdf]

9.3 Implementacje referencyjne

- [10] **Moonpuck.** *chameleon_cluster — Python implementation of CHAMELEON algorithm*. Repozytorium GitHub, commit 1c0a65ee6a79706e4d415dd7ca78da5d3c29906d. https://github.com/Moonpuck/chameleon_cluster [implementacja referencyjna używana do walidacji — patche w scripts/setup_reference.sh]

9.4 Biblioteki i narzędzia

- [11] **Harris C.R., Millman K.J., van der Walt S.J. et al.** *Array programming with NumPy*. Nature, vol. 585, 2020, str. 357–362. [biblioteka numpy]

- [12] **Virtanen P., Gommers R., Oliphant T.E. et al.** *SciPy 1.0: fundamental algorithms for scientific computing in Python*. Nature Methods, vol. 17, 2020, str. 261–272. [`scipy.spatial.KDTree` — Faza 0]
- [13] **Pedregosa F., Varoquaux G., Gramfort A. et al.** *Scikit-learn: Machine learning in Python*. JMLR, vol. 12, 2011, str. 2825–2830. [interfejs `BaseEstimator/ClusterMixin/check_estimator`]
- [14] **Klockner A.** *PyMETIS: Python wrapper for METIS graph partitioning library*. Repozytorium PyPI: <https://pypi.org/project/PyMetis/> [wrapper METIS używany w Fazie I]
- [15] **van Rossum G., Lehtosalo J., Langa Ł.** *PEP 484: Type Hints*. Python Enhancement Proposal, 2014. [system type hints używany w `_types.py`]
- [16] **Astral.** *uv: An extremely fast Python package and project manager*. <https://github.com/astral-sh/uv> [menedżer pakietów używany w projekcie]
- [17] **Astral.** *ruff: An extremely fast Python linter and code formatter*. <https://github.com/astral-sh/ruff> [linter w pre-commit i CI]
- [18] **Lehtosalo J. et al.** *mypy: Optional static typing for Python*. <http://mypy-lang.org/> [type checker w trybie strict]